The Definitive Guide to WebXR Adoption (Mid-2026 Edition)

Prepared for Gregory L. / Professor Codephreak — for the PYTHAI / DeltaVerse "BubbleRoom" immersive environment

TL;DR

- **WebXR is production-viable in 2026 but still a Candidate Recommendation Draft, not a finished W3C standard.** The pragmatic stack for the DeltaVerse BubbleRoom is **Three.js + `@react-three/xr` v6** rendered over HTTPS, progressively enhanced from desktop 3D → mobile "magic window" AR → immersive VR, targeting Meta Quest Browser (the most complete WebXR target), Android XR / Samsung Galaxy XR, and Apple Vision Pro (VR-only).
- **Token-gating (OVERLORD) must happen BEFORE the immersive session begins**, because opening a MetaMask/WalletConnect popup destroys an active `XRSession`. Gate entry with EIP-4361 Sign-In with Ethereum at the 2D page, mint a server session/JWT, then call `requestSession()` — and consider x402 (including the Algorand AVM implementation by GoPlausible) for pay-per-scene access.
- **The ecosystem consolidated sharply: Mozilla Hubs (2024) and 8th Wall (winding down through 2027) are both gone or going.** Self-host on open-source foundations — Hyperfy, networked-aframe, Hubs Community Edition, Multisynq/Croquet — which aligns with your Podman + IPFS/Arweave self-hosted infrastructure.

Key Findings

1. **Spec status.** The WebXR Device API is a **Candidate Recommendation Draft** (latest CRD dated 2026-03-16 at w3.org/TR/webxr/), edited by Brandon Jones and Manish Goregaokar (Google) and Rik Cabanier (Meta). It needs two independent interoperable implementations passing the test suite to advance to Proposed Recommendation. Core repo: github.com/immersive-web/webxr.
2. **Browser reality.** WebXR ships in Chromium browsers (Chrome/Edge 79+, Opera, Samsung Internet 12+), the Meta Quest Browser (most complete), Android XR's Chrome, and **Safari on visionOS 2+ (VR only — no AR module)**. It is **absent from desktop Safari, iOS/iPadOS Safari, and Firefox**. caniuse rates it ~50/100 cross-browser.
3. **Framework of choice.** Three.js has first-class built-in WebXR (`renderer.xr` / `WebXRManager`), and `@react-three/xr` v6 (a full rewrite by pmndrs, maintainer Bela Bohlender) is the most ergonomic React path with built-in IWER emulation. This maps directly onto your existing React Three Fiber DeltaVerse stack.
4. **OVERLORD = pre-session auth.** Your NeuralNode contracts (BubbleRoomV4 ERC-721 with role-based access, DeltaGenesisSBT soulbound identity, DeltaVerseOrchestrator EIP-

712 gasless minting) should gate at the 2D entry page via SIWE, NOT inside the headset. This avoids the popup-kills-session problem entirely.

Details

1. The WebXR Device API & the immersive-web modules

The **WebXR Device API** (w3.org/TR/webxr/, repo github.com/immersive-web/webxr) is the core. The "X" is a variable standing for VR/AR/MR. It exposes three session modes: `inline` (magic-window in a normal page), `immersive-vr`, and `immersive-ar`. All immersive sessions require a secure context (HTTPS) and a user gesture.

The Immersive Web Working Group (final specs) and Immersive Web Community Group (incubation) maintain all modules under **github.com/immersive-web**. Module status as of mid-2026:

- **WebXR Augmented Reality Module** — CR (github.com/immersive-web/webxr-ar-module). Maintained, no major new features planned. Adds `immersive-ar` + environment blend modes.
- **Hit Test Module** — github.com/immersive-web/hit-test. Stable; places content on real-world geometry.
- **Anchors Module** — github.com/immersive-web/anchors. Poses that stay aligned to the physical world within a session.
- **Hand Input Module** — github.com/immersive-web/webxr-hand-input. Exposes 25 joint poses per hand.
- **Layers API** — github.com/immersive-web/layers. Composition layers (quad, cylinder, equirect, cube, projection) for sharper text/video and lower power.
- **Depth Sensing** — github.com/immersive-web/depth-sensing. Real-world depth/occlusion (Quest Depth API enables instant MR placement without a pre-scanned mesh).
- **Raw Camera Access** — github.com/immersive-web/raw-camera-access.
- **DOM Overlay** — github.com/immersive-web/dom-overlays. 2D HTML over an immersive-ar session (handheld AR).
- **Lighting Estimation** — github.com/immersive-web/lighting-estimation.
- **Plane Detection** — github.com/immersive-web/real-world-geometry (plane detection portion).
- **Mesh Detection** — github.com/immersive-web/real-world-meshing.
- **WebXR/WebGPU Binding Module** — Editors' Draft, immersive-web.github.io/WebXR-WebGPU-Binding (repo github.com/immersive-web/WebXR-WebGPU-Binding). Important for the future.

The live capability matrix at **immersiveweb.dev** reads modules off your current browser. Feature detection is mandatory: always pass modules in `optionalFeatures` and check `session.enabledFeatures`. To embed WebXR in an iframe you must add `allow="xr-spatial-tracking"`.

2. Browser & device landscape (mid-2026)

Browser / Device	WebXR status
Chrome / Edge (Android & desktop)	Full VR + AR module on capable hardware; the reference implementation.
Meta Quest Browser (Horizon OS)	Most complete: passthrough AR, hand tracking, anchors, plane detection, hit test, Depth API, Three.js Resources WebGL projection layers, OCULUS_multiview.
Apple Vision Pro / visionOS Safari	**WebXR enabled by default since visionOS 2** (immersive-vr only). Uses the gaze-and-pinch **transient-pointer** input mode (added to the spec by Apple). **No `immersive-ar`** — the AR flag exists but is non-functional per Apple's own engineers. A visionOS 26.5 "WebXR Augmented Reality Module" Safari flag appeared but does not yet produce a working AR session.
Android XR / Samsung Galaxy XR	Samsung Galaxy XR (codename "Project Moohan") was officially unveiled **October 21, 2025 at US$1,799.99**, with shipments beginning **October 31, 2025**. Per Road to VR: "dual 3,552 x 3,840 Micro-OLED displays clocked at a default 72Hz refresh (max 90Hz), 256GB of storage, 16GB of RAM and Qualcomm Snapdragon XR2+ Gen 2 chipset" (motion controllers sold separately for $250). Supports WebXR via Chrome / Samsung Internet on Android XR.
Samsung Internet 12+	Supported, optimized for Galaxy devices and Galaxy XR.
Wolvic	The only open-source WebXR browser for standalone Android/AOSP headsets (Quest, Pico, HTC, Lynx, Huawei). Gecko- and Chromium-based builds; has a kiosk mode useful for locked-down BubbleRoom deployments.
Pico browser	Limited native WebXR; Wolvic is the recommended path.
iOS / iPadOS Safari	No WebXR. Workarounds historically: the WebXR Viewer app (Mozilla, now unmaintained) and Variant Launch (8th Wall's SDK to bring WebAR to iOS) — but 8th Wall is winding down.

Browser / Device	WebXR status
Desktop Safari / Firefox	No WebXR.
Desktop VR (SteamVR via Chrome)	Works through Chrome's OpenXR/OpenVR path for tethered headsets.

Igalia implemented WebXR in WebKit for the WPE/GTK ports during 2025 (including Hand Input by August 2025 and the AR module), which is the engine path that could eventually broaden Linux WebXR support — relevant to your self-hosted Podman/Linux infrastructure.

3. Frameworks & libraries (with repos)

- **Three.js** (github.com/mrdoob/three.js) — built-in WebXR via `renderer.xr.enabled = true`. Key addons in `examples/jsm/webxr/`: `VRButton.js`, `ARButton.js`, `XRButton.js`, `XRControllerModelFactory.js`, `XRHandModelFactory.js`, `XREstimatedLight.js`, `XRPlanes.js`. Live examples list: threejs.org/examples/ (filter "webxr"). Multiview (OCULUS_multiview) and WebGPU+WebXR multiview have landed via PRs by Rik Cabanier (e.g. PR #30920).
- **React Three Fiber** + **`@react-three/xr` v6** (github.com/pmndrs/xr) — v6 is a complete rewrite aligning with the R3F event system; `createXRStore()`, `<XR>`, `store.enterVR()/enterAR()`, `offerSession`, and **built-in IWER + @iwer/devui emulation on localhost**. This is the recommended path for the DeltaVerse React stack.
- **Babylon.js** (github.com/BabylonJS/Babylon.js) — `WebXRDefaultExperience`, `WebXRFeaturesManager`; hand tracking on by default when supported (25 joints); recently added WebXR Body Tracking (83 joints).
- **A-Frame** (github.com/aframevr/aframe) — declarative HTML ECS on top of Three.js; the basis of Mozilla Hubs and networked-aframe.
- **PlayCanvas** (github.com/playcanvas/engine) — WebGL/WebGPU engine with WebXR.
- **Wonderland Engine** (wonderlandengine.com) — WASM-compiled, high-performance, native editor. Per GamesBeat (Jan 29, 2025), "Wonderland and Robot Invader revealed that their two companies have merged, creating a new company called Create Worlds," combining Wonderland Engine and Robot Invader's Story Machine; Wonderland founder Jonathan Hale became Create Worlds CEO. Still actively shipping (newsletter Jan 2026: built-in locomotion, OPUS audio).
- **Needle Engine** (github.com/needle-tools, docs.needle.tools) — "Three.js with superpowers"; Unity/Blender exporter to glTF/GLB web components with networking + XR.

- **model-viewer** (github.com/google/model-viewer) — `<model-viewer>` web component with AR Quick Look / Scene Viewer.
- **AR.js** (github.com/AR-js-org/AR.js) — marker/location AR.
- **8th Wall — ⚠️ winding down.** Per 8th Wall's official transition FAQ: "February 28, 2026: End of platform access... February 28, 2026 – February 28, 2027: Existing published/hosted projects continue to function, but users cannot log in or modify their projects. After February 28, 2027: 8th Wall hosting services will be decommissioned, and remaining project data deleted." (Niantic acquired 8th Wall in 2021.) Commercial license had dropped to $700/project/month (a 75% cut from $3,000) on Sept 4, 2024, before the shutdown was announced. Migrate off it.
- **Zappar / Mattercraft** (zap.works) — WebAR authoring; still commercial.
- **Godot WebXR export** — WebXR merged into Godot core (3.4+) by David Snopek; fully supported in Godot 4.x via the `WebXRInterface` class (docs.godotengine.org/en/stable/classes/class_webxrinterface.html — "XR interface using WebXR... WebXR is an open standard that allows creating VR and AR applications that run in the web browser"). Companion: **godot-xr-tools** (github.com/GodotVR/godot-xr-tools, by Bastiaan Olij), latest line XR Tools 4.4.0 for Godot 4.2+. Community WebXR template: github.com/msub2/godot-webxr-template.
- **Unity WebXR export — De-Panther/unity-webxr-export** (github.com/De-Panther/unity-webxr-export), Apache 2.0, actively maintained (commits through Oct 2025, ~1.2k stars, 59 release tags). README: "WebXR Export supports both Augmented Reality and Virtual Reality WebXR API immersive sessions." Supports Unity 2020.3.11f1 → Unity 6 (6000.0.23f1+); WebXR Device API, Gamepads, Hit Test, and **Hand Input via the Unity XR Hands package**. OpenUPM packages `com.de-panther.webxr` and `com.de-panther.webxr-interactions`. Tested on Quest Browser/Wolvic, Vision Pro Safari, HoloLens 2 Edge, Android Chrome/Samsung Internet, Pico 4 (Wolvic), Magic Leap 2, and Snap Spectacles.
- **WebXR Polyfill** (github.com/immersive-web/webxr-polyfill) — fallback for older browsers (largely superseded by IWER for emulation).
- **Emulators — Meta's Immersive Web Emulator** browser extension (github.com/meta-quest/immersive-web-emulator) and its runtime **IWER** (github.com/meta-quest/immersive-web-emulation-runtime), with `@iwer/sem` (synthetic environments) and `@iwer/devui`. IWER v2 replaced the old WebXR-polyfill fork and now works with `@react-three/xr`. The older **Mozilla WebXR emulator** is effectively superseded. GitHub

4. Hand tracking, controllers & input

- **WebXR Hand Input Module** (github.com/immersive-web/webxr-hand-input) exposes 25 joint `XRJointSpace` poses per hand; in Three.js via `renderer.xr.getHand(i)` + `XRHandModelFactory`.

- **Controllers / gamepad mapping — github.com/immersive-web/webxr-input-profiles** holds the asset library + JSON schema for rendering correct controller models and standardizing button/axis layout. Three.js `XRControllerModelFactory` consumes these profiles.
- **Apple Vision Pro input** — gaze-and-pinch via the **transient-pointer** mode; a `select` event fires on pinch. No persistent gaze stream (privacy). Design BubbleRoom interactions around discrete pinch selects, not hover/dwell.
- **Hand pose / gesture libraries** — Babylon.js gesture recognizers; MediaPipe/TensorFlow.js for non-WebXR camera hand tracking on phones.

5. Performance & best practices

- **WebXR Layers API** (github.com/immersive-web/layers) — use quad/cylinder/equirect layers for UI, text, and video so the compositor renders them at native resolution (huge legibility win for token/wallet UI panels in-headset). Chrome added WebGL `XRProjectionLayer` support; Quest has had layers for a while.
- **Foveation** — `renderer.xr.setFoveation(1)` in Three.js / `foveation` prop in R3F (0 = full res, 1 = max edge reduction).
- **Framebuffer scaling** — `XRWebGLLayer` `framebufferScaleFactor`; query recommended resolution and scale down for fill-rate-bound scenes.
- **Multiview — OVR_multiview2 / OCULUS_multiview** (adds multisampling) render both eyes in one pass via `gl_ViewID_OVR`. Meta documents this as the recommended Quest rendering path (developers.meta.com/horizon/documentation/web/web-multiview/). Create the context with `antialias: false` for OVR_multiview2; use OCULUS_multiview for MSAA.
- **WebGPU + WebXR** — the **WebXR/WebGPU Binding Module** (immersive-web.github.io/WebXR-WebGPU-Binding) provides `XRGPUBinding` from an `xrCompatible` adapter. Brandon Jones shipped experimental Chrome support (Canary 135+, flags "WebXR Projection Layers" + "WebXR/WebGPU Bindings") in March 2025; not yet an automatic perf win (an internal texture copy remains). Three.js WebGPURenderer multiview support landed via Cabanier's PRs. WebGPU's one-texture-per-eye model is also why Apple is pursuing WebGPU bindings for Vision Pro performance.
- **Rendering budgets** — for standalone headsets keep total scene ⩽ ~100k triangles, use `instancedMesh`, avoid per-frame allocations in the render loop, and target 72–90 Hz.
- **Asset pipeline** — glTF 2.0 + **Draco** (geometry), **KTX2/Basis Universal** (GPU textures), **meshopt** (compression). These are essential for IPFS-hosted assets where transfer size dominates load time.

6. Multiplayer / networked immersive spaces (BubbleRooms)

- **networked-aframe** (github.com/networked-aframe/networked-aframe) —

WebRTC/WebSocket multi-user A-Frame; adapters for EasyRTC, Janus (SFU), Firebase. Mature, MIT (~1,200 stars).

- **Mozilla Hubs — shut down by Mozilla May 31, 2024** (announced Feb 15, 2024 amid an org-wide restructuring). Per Mozilla Support: "Mozilla has ended support for Mozilla Hubs on May 31st, 2024… The Hubs codebase is now being managed and maintained by the Hubs Foundation." **Hubs Community Edition** is self-hostable on any Kubernetes platform — a viable open foundation but requires K8s expertise.
- **Hyperfy** (github.com/hyperfy-xyz/hyperfy) — open-source (built on Three.js + PhysX), self-hosted persistent worlds, in-world JS app system, WebXR + VRM avatars, and **native Ethereum/blockchain integration for token-gated worlds and 3D NFTs**. The closest off-the-shelf match to the BubbleRoom + OVERLORD concept; worth forking.
- **Colyseus** (github.com/colyseus/colyseus) — authoritative Node.js room-based multiplayer server (good for swarm/consensus-style server logic).
- **Croquet / Multisynq** (github.com/croquet/croquet, github.com/multisynq/multisynq-client) — synchronized client-side VMs, no server-side game code; Multisynq runs on a DePIN network with end-to-end encryption — philosophically aligned with PYTHAI's decentralization. An A-Frame Croquet component exists.
- **WebRTC spatial audio** — position-attenuated audio via Janus SFU (networked-aframe) or PannerNode-based Web Audio graphs synced to avatar transforms.
- **Other open platforms** — Overte (open High Fidelity successor), WorkAdventure, and the curated `awesome-webxr` / `awesome-metaverse` lists (github.com/msub2/awesome-webxr, github.com/M3-org/awesome-metaverse).

7. WebXR + blockchain / token-gating (OVERLORD)

The core problem: launching a wallet popup (MetaMask extension, WalletConnect modal) blurs/suspends the page and **ends the active** `XRSession`. Therefore:

Pattern A — Pre-session gate (recommended for OVERLORD). Authenticate and verify token ownership on the **2D entry page** before `requestSession()`:

1. User connects wallet on the flat page.
2. **EIP-4361 Sign-In with Ethereum** (eips.ethereum.org/EIPS/eip-4361; reference impl github.com/spruceid/siwe): server issues a nonce, user signs the structured message, server verifies, mints a session cookie/JWT. The signed message binds `domain`, `address`, `chain-id`, `nonce`, `issued-at`.
3. Server checks BubbleRoomV4 role / token balance (your `BubbleRoomV4` ERC-721 with role-based access — this IS the OVERLORD gate).
4. Only then enable the "Enter VR/AR" button → `requestSession()`. The headset session never needs the wallet again.

Pattern B — Session keys. Pre-authorize a temporary, scoped key (or a smart-account session key) on the 2D page so any in-world transactions (spawning rooms, trait accrual via your EmergenceTraits contract) are signed silently without a popup. Pairs well with your `DeltaVerseOrchestrator` EIP-712 gasless minting.

Pattern C — x402 pay-per-scene. The **x402** protocol (HTTP 402, Coinbase-originated) lets a server return `402 Payment Required` and gate a resource until paid — ideal for paying to load a premium BubbleRoom scene or asset. The **Algorand AVM implementation by GoPlausible** (x402.goplausible.xyz; docs under github.com/GoPlausible/.github) ships scoped npm packages `@x402-avm/core`, `@x402-avm/avm`, `@x402-avm/express`, `@x402-avm/hono`, `@x402-avm/next`, and a Python `x402-avm` package, plus a `@x402-avm/paywall` that connects Pera/Defly/Lute wallets via `@txnlab/use-wallet`. It also has an AP2-x402 (Google Agent Payments) extension and a "Bazaar" discovery extension — directly relevant to AgenticPlace/mindX agents paying for XR content autonomously. (The "Parsec / parsec-wallet" naming in the original brief was **not** confirmed in public repos; the confirmed public Algorand x402 work lives under GoPlausible's `x402-avm` packages and facilitator.)

Token-gated 3D-space precedents: Decentraland, Voxels (formerly Cryptovoxels), **Hyperfy** (open-source, on-chain world ownership), Substrata (open-source, substrata.info), Webaverse (now "Upstreet"; original webaverse repos archived), Mona, Spatial.io, oncyber.

ERC-8004 (Trustless Agents) — eips.ethereum.org/EIPS/eip-8004, **Draft** (created 2025-08-13), authors Marco De Rossi, Davide Crapis (Ethereum Foundation), Jordan Ellis (Google), Erik Reppel (Coinbase). It "proposes to use blockchains to discover, choose, and interact with agents across organizational boundaries without pre-existing trust, thus enabling open-ended agent economies," via three on-chain registries — **Identity** (ERC-721-based, URIStorage), **Reputation**, and **Validation**. It is **not XR-specific** (the spec makes no mention of XR), but it is a natural identity/trust backbone for the MASTERMIND/GENESIS/CHRONOS/KAIROS/BUILDER agents in your SeedRegistry/SwarmGovernance system if they need portable, verifiable identity across spaces. Treat the "live on mainnet" claims circulating in early 2026 as unverified — the canonical EIP is still Draft.

8. Practical adoption roadmap for PYTHAI

Progressive enhancement ladder:

1. **Desktop 3D** — ship the BubbleRoom as a normal Three.js/R3F scene (your existing `deltaverse-point-of-departure` R3F showcase fork is a good seed). Works everywhere, no XR needed.
2. **Mobile AR** — add `immersive-ar` with hit-test + DOM overlay for Android Chrome "magic window" placement.
3. **Immersive VR** — add `immersive-vr`, controllers, hands, teleport locomotion for Quest / Galaxy XR / Vision Pro.

Three.js skeleton:

```js
import * as THREE from 'three';
import { VRButton } from 'three/addons/webxr/VRButton.js';
import { XRControllerModelFactory } from 'three/addons/webxr/XRControllerModelFactory

const renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
renderer.xr.enabled = true;
renderer.xr.setFoveation(1);
document.body.appendChild(renderer.domElement);
document.body.appendChild(VRButton.createButton(renderer, {
  optionalFeatures: ['local-floor', 'bounded-floor', 'hand-tracking', 'layers']
}));

const ctrlFactory = new XRControllerModelFactory();
for (let i = 0; i < 2; i++) {
  const grip = renderer.xr.getControllerGrip(i);
  grip.add(ctrlFactory.createControllerModel(grip));
  scene.add(grip);
}
renderer.setAnimationLoop(() => renderer.render(scene, camera));
```

R3F equivalent: `createXRStore()` + `<XR store={store}>` + `store.enterVR()` (gate the button behind your SIWE check). Start from Meta's tutorials: github.com/meta-quest/webxr-first-steps and …/webxr-first-steps-react.

Testing without hardware: Meta Immersive Web Emulator extension + IWER (built into `@react-three/xr` on localhost). IWER's `@iwer/sem` emulates planes/meshes/anchors; ActionRecorder/ActionPlayer replay real Quest sessions.

Smart contracts / OVERLORD: Your NeuralNode suite is already Foundry-tested (Solidity 0.8.24, 44 tests, 9 contracts — BubbleRoomV4, DeltaGenesisSBT, DeltaVerseOrchestrator, BubbleRoomSpawn, EmergenceTraits, SeedRegistry, SwarmGovernance, TombRegistry). For mainnet: pin Solidity + OpenZeppelin versions, run `forge test` + fuzz + invariant tests, get an audit before gating real value, use a multisig (you already fork Gnosis Safe modules/zodiac) as contract owner, and verify on the explorer. Gate entry by reading `BubbleRoomV4` role/balance server-side after SIWE.

Serving / deployment (Podman + IPFS):

- **HTTPS is mandatory** for any immersive session — terminate TLS at your reverse proxy (Caddy/nginx) in the Podman pod.

- **Permissions-Policy headers** — send `Permissions-Policy: xr-spatial-tracking=(self)`; for embeds use iframe `allow="xr-spatial-tracking"`. Add `camera`/`microphone` directives if using raw camera / spatial voice.
- **IPFS/Arweave glTF hosting** — pin Draco/KTX2-compressed glTF to IPFS (you already fork decentralized-websites, deltastorage); serve via a gateway with correct **CORS** (`Access-Control-Allow-Origin`) and MIME types (`model/gltf-binary`). Cross-origin assets need CORS or Three.js GLTFLoader will fail. Consider COOP/COEP headers (`Cross-Origin-Opener-Policy: same-origin`, `Cross-Origin-Embedder-Policy: require-corp`) if you use `SharedArrayBuffer` (multithreaded Draco/Basis) — but note COEP can complicate loading cross-origin IPFS assets, so set `crossorigin` on loaders and configure gateway CORS accordingly.

9. Learning resources & reference projects

- **MDN WebXR** — developer.mozilla.org/en-US/docs/Web/API/WebXR_Device_API.
- **web.dev / Chrome** — WebXR articles and the toji.dev blog (Brandon "Toji" Jones) for WebGPU+WebXR.
- **Immersive Web Samples** — github.com/immersive-web/webxr-samples (live: immersive-web.github.io/webxr-samples/), including WebGPU barebones examples.
- **Three.js examples** — threejs.org/examples/ (webxr_*).
- **Meta WebXR First Steps** — github.com/meta-quest/webxr-first-steps and …/webxr-first-steps-react (2-hour tutorials).
- **awesome-webxr** — github.com/msub2/awesome-webxr (curated tools, games, experiences: Moonrider, Hello WebXR, Plockle, Castle Builder, etc.).
- **immersiveweb.dev** — live capability matrix + explainer.

Recommendations

Stage 1 (now): Build the BubbleRoom on **Three.js** + `@react-three/xr` **v6** as a desktop-first R3F scene. Wire OVERLORD as a **server-side SIWE (EIP-4361) gate** that reads `BubbleRoomV4` role/balance, issuing a JWT before the "Enter" button activates. Test entirely in the **Immersive Web Emulator / IWER**. *Benchmark to advance:* a gated scene that loads a Draco/KTX2 glTF from your IPFS gateway and runs ⩾72 fps in the emulator.

Stage 2: Add `immersive-vr` for **Meta Quest 3/3S** (your primary target — most complete WebXR). Add controllers (webxr-input-profiles), hand tracking, teleport locomotion, and **WebXR Layers** for wallet/token UI panels. Adopt **OVR/OCULUS_multiview** + foveation. *Benchmark:* stable 72–90 Hz on-device with multiview enabled.

Stage 3: Add **mobile AR** (`immersive-ar` + hit-test + DOM overlay) for Android Chrome, and validate **Galaxy XR** and **Vision Pro (VR-only)**. Use feature detection, never user-agent sniffing.

Stage 4 (multiplayer): Make BubbleRooms social. Fork **Hyperfy** (closest to your token-gated-world + 3D-NFT model) or build on **networked-aframe + Janus** or **Multisynq**. Add WebRTC spatial audio.

Stage 5 (agentic commerce): If agents (MASTERMIND/CHRONOS) or users should pay to enter/spawn premium rooms, integrate **x402** — GoPlausible's `@x402-avm/*` for Algorand, or Coinbase's EVM x402 — and evaluate **ERC-8004** for portable agent identity. Keep these behind the same pre-session gate.

Thresholds that change the plan: If/when **visionOS ships a working WebXR AR module**, add Vision Pro AR. If **WebGPU+WebXR** exits experimental in Chrome/Quest, migrate the renderer to Three.js WebGPURenderer for compute-shader-driven effects. If you depend on any **8th Wall** content, export it before **Feb 28, 2027**.

Caveats

- **Spec is still a Candidate Recommendation Draft**, MDN flags WebXR "experimental," and module support is fragmented — **feature-detect everything**.
- **Vision Pro is VR-only**; do not promise passthrough AR BubbleRooms on Apple hardware yet.
- **No WebXR on iOS/iPadOS Safari, desktop Safari, or Firefox** — these users get the desktop-3D / magic-window fallback only.
- **Accessibility is largely unsolved** — canvas scenes are opaque to screen readers; there is no standardized way to expose scene structure to assistive tech.
- **The "Parsec / parsec-wallet" Algorand x402 naming could not be verified** in public repos; the confirmed public work is GoPlausible's `x402-avm`. **ERC-8004 is Draft**, and "live on mainnet" claims are from secondary/community sources only.
- **Ecosystem churn is real:** Mozilla Hubs is gone, 8th Wall is winding down (access ends Feb 28, 2026; hosting decommissioned Feb 28, 2027), and Wonderland merged into Create Worlds — plan for self-hosting on open foundations, which fits your Podman/IPFS strategy.
- WebXR adoption/"40% growth in 2026" figures circulating in trade press (vr.org) are marketing-flavored estimates, not audited data — treat directionally.